

MPMC PROJECT REPORT

MPMC PROJECT

S Nandhish Kumar (110124118)

—

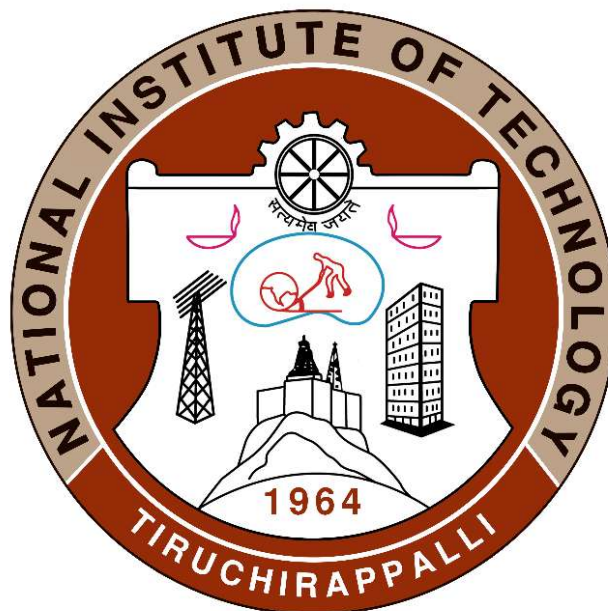
Sivdev Pandian (110124116)

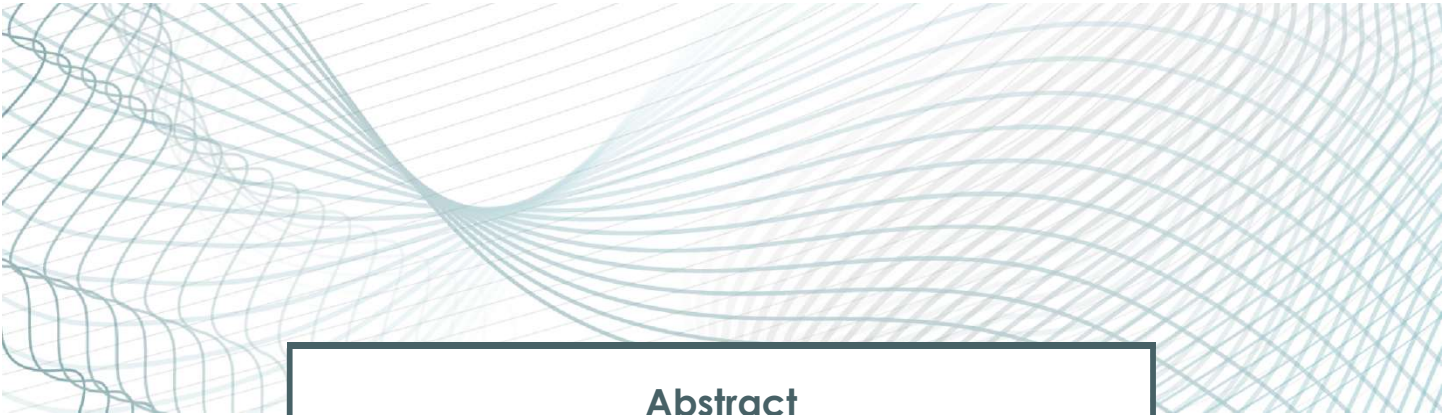
—

Soubhagya (110124120)

TITLE

Design and Implementation of an
IMU-Controlled Wi-Fi Rover using
ESP32 and Differential Kinematics





Abstract

This project details the build of a wireless bot controlled entirely by tilting a smartphone.

Instead of using a standard joystick, the system gets pitch and roll data from the phone's built-in gyroscope.

The main controller is an ESP32, which acts as a local web server and communicates with the phone over Wi-Fi using standard HTTP requests.

The ESP32 translates the tilt angles into PWM signals to drive two BO motors via an L298N motor driver using differential steering.

A major focus of the software was handling real-world hardware limits—specifically, writing JavaScript filters to ignore hand tremors, clamping the math to prevent integer overflow, and coding a "kickstart" voltage jump to break the static friction of the plastic gearboxes.

Network traffic was also hard-capped at 4Hz to prevent the ESP32 from crashing, resulting in a smooth, proportional driving experience.

Hardware Architecture

This bot is built using three main hardware parts: microcontroller, the motor driver, and the motors.

1. ESP32 Microcontroller : The ESP32 handles the communication and the math. It does two main jobs:

- **Wi-Fi Server:** It creates its own local Wi-Fi network. When you tilt your phone, the phone sends the target speeds (like $l=150$ and $r=80$) directly to the ESP32 over this Wi-Fi.
- **Logic Control:** It reads those text numbers, converts them into actual math, and outputs PWM (Pulse Width Modulation) signals through its pins to control the motor speeds.

2. L298N Motor Driver: The ESP32 pins only output 3.3V and very little current. If you connect a motor directly to it, the ESP32 will burn out. We use the L298N driver as a middleman.

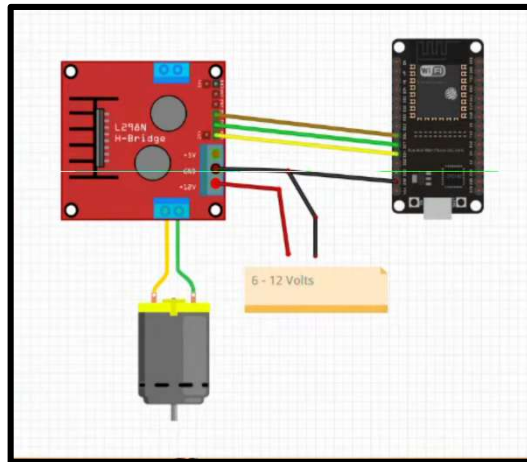
- The L298N takes the low-power PWM signal from the ESP32 and uses the heavy current from the battery pack to actually spin the motors.
- It also contains an H-Bridge circuit. This allows us to flip the positive and negative power going to the motors so the rover can drive in reverse.

3. The Wheels: BO Motors We use two standard Battery Operated (BO) gear motors, one on each side.

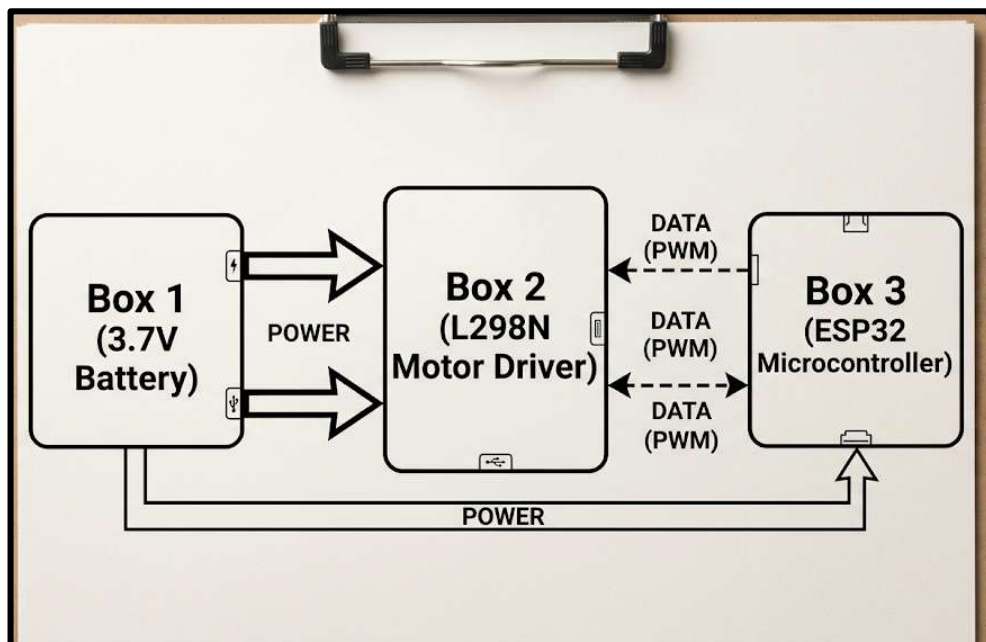
- **Steering:** To turn right, we spin the left wheel faster than the right wheel. To spin in place, one-wheel drives forward while the other reverses.
 - **The Friction Problem:** The cheap plastic gears inside these motors have a lot of static friction. If we send a low-speed signal (like 20 or 30 PWM), the motors just hum and get stuck. To fix this, the software is designed to "kickstart" the motors with a minimum of 80 PWM so they always have enough force to break friction and start moving.
-

Hardware Architecture

Circuit diagram (For connecting a single motor)



Hardware block diagram



1. The Phone (Frontend & JavaScript)

- **Reading the Sensor:** The code constantly monitors the phone's physical gyroscope to check how far you are tilting it forward (pitch) and twisting it sideways (roll).
- **The Math:** It takes those tilt angles and converts them into motor speeds. It includes a "deadzone" so that if your hand is just shaking slightly, the rover ignores it. It also adds an automatic "kickstart" (jumping straight to 80 speed) so the motors don't get stuck.

2. The Communication (Wi-Fi & HTTP)

Once the phone calculates the exact left and right motor speeds, it needs to send them to the ESP32.

- It packages the numbers into a simple web address format, like `"/drive?l=150&r=80"`.
- **Speed Limit:** The phone's sensor updates extremely fast, but if it sent hundreds of messages a second, the ESP32 would crash. The JavaScript is programmed to intentionally pause and only send a message 4 times per second (4Hz) to keep the connection perfectly stable.

3. The ESP32 (Backend & C++)

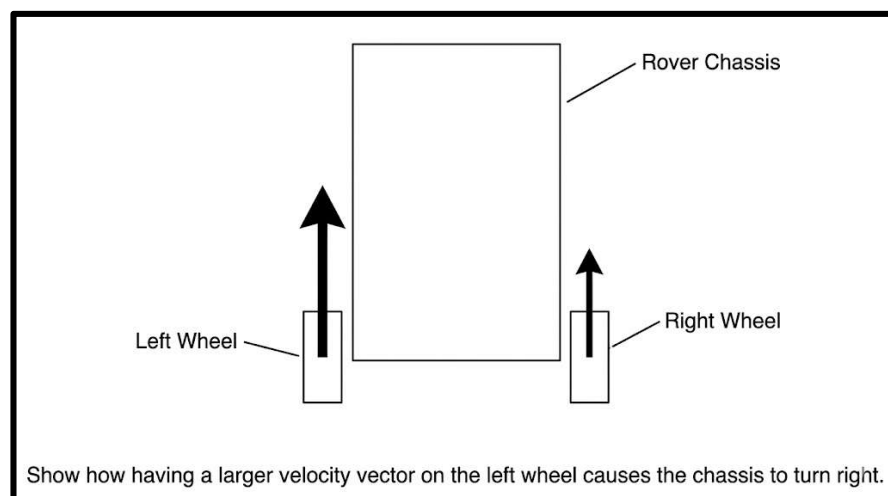
- **The Web Server:** The ESP32 constantly listens on Port 80. When it receives that `/drive` web address from the phone, it catches it.
- **Translation:** The message arrives as pure text. The C++ code scans the text, finds the numbers for the left and right motors, and translates them from text characters into actual math numbers.
- **Action:** Finally, it hands those pure numbers to the hardware pins, sending the PWM signals to the L298N to physically spin the wheels.

Control theory & Kinematics

1. Differential Kinematics (The Mixer) Because the rover does not have a steering wheel, it relies on "differential steering." The JavaScript takes the forward tilt (Throttle) and the side-to-side twist (Steering) and mixes them using simple algebra:

- Left Motor Speed = Throttle + Steering
- Right Motor Speed = Throttle - Steering

2. Signal Conditioning & The Dead zone Raw sensor data is noisy. If you hold your hand "perfectly still," the gyroscope is still reading micro-movements. The software includes a 15-degree deadzone for the throttle and a 10-degree deadzone for the steering.



3. Signal Saturation (The Clamp) The motor driver relies on an 8-bit PWM signal, meaning it can only accept speed numbers between 0 and 255. If you tilt the phone extremely hard forward and to the side simultaneously, the Throttle + Steering math might calculate a speed of 300. Sending 300 to the ESP32 will cause an integer overflow, resulting in unpredictable behavior. To prevent this, the final step of the algorithm is a strict mathematical clamp.

Conclusion

This project successfully demonstrated the design and implementation of a wireless telemetry rover controlled by human wrist kinematics.

By utilizing an ESP32 as a local web server, the system effectively bridged the gap between a smartphone's MEMS sensors and physical hardware actuation over a standard TCP/IP network.

The core achievement of this build was resolving mechanical and networking hardware limitations entirely through software logic.

By implementing a 15-degree dead zone, an 80 PWM kickstart, and strict saturation clamping, the JavaScript control system successfully masked the inherent static friction of the BO gearboxes and prevented integer overflow at extreme tilt angles.

Additionally, artificially throttling the network transmission rate to 4Hz ensured the ESP32 did not suffer from buffer overruns, resulting in a stable, low-latency differential drive system without the need for a dedicated radio transmitter.